

Solve the Schrödinger equation

```
In [1]: import numpy as np
import matplotlib.pyplot as plt
from scipy.integrate import trapezoid
```

```
In [2]: # Define the Schrödinger equation based on the given potential (atomic
units)
# Stationary Schrödinger equation:  $-(\hbar^2/2m) \psi'' + V \psi = E \psi$ 
# In atomic units  $\hbar = m = 1$ , so  $-1/2 \psi'' + V \psi = E \psi$ 
# Rearranged:  $\psi'' = 2(V - E) \psi$ 
#  $V = V_0 x/x_0$  for  $0 < x < x_0$ 
#  $V = V_0$  elsewhere
#  $x_0 = 5$ ,  $V_0 = 50$ 
# Split into two first-order equations:  $u' = v$ ,  $v' = 2(V - E)u$ 
 $x_0, V_0 = 5, 50$ 

# RK4 function
# E: trial energy
# xa, xb: start and end point of the integration
# u, v: psi and psi' at the start point
# n: number of steps
# Returns the x points, psi at every point, and psi' at the end point
def rk4(E, xa, xb, u, v, n=1000):
    h = (xb - xa)/n # step size, negative if we go from right to left
    x = np.linspace(xa, xb, n+1)
    us = [u] # save psi at every step
    g = lambda x, u, v: (v, 2*(V_0*x/x_0 - E)*u) # right-hand side:  $u' = v$ ,
 $v' = 2(V - E)u$ 
    for xi in x[:-1]:
        # RK4: slope at the start (k1), twice in the middle (k2, k3) and at
the end (k4)
        k1u, k1v = g(xi, u, v)
        k2u, k2v = g(xi + h/2, u + h/2*k1u, v + h/2*k1v)
        k3u, k3v = g(xi + h/2, u + h/2*k2u, v + h/2*k2v)
        k4u, k4v = g(xi + h, u + h*k3u, v + h*k3v)
        # Next value = current value + weighted average of the slopes
(weights 1, 2, 2, 1)
        u += h/6*(k1u + 2*k2u + 2*k3u + k4u)
        v += h/6*(k1v + 2*k2v + 2*k3v + k4v)
        us.append(u)
    return x, np.array(us), v
```

Outside the well $V = V_0$, so ψ is an exponential there. Same as section 4.9 in the book, I integrate from both sides and match them at the turning point, where $V = E$. If I only integrate from left to right, the solution blows up where ψ should decay.

```
In [3]: # Left side:  $\psi = e^{kx}$  outside the well, so  $\psi = 1$ ,  $\psi' = k$  at  $x = 0$ 
# Right side:  $\psi = e^{-kx}$  outside the well, so  $\psi = 1$ ,  $\psi' = -k$  at  $x = x_0$ 
#  $k = \sqrt{2(V_0 - E)}$ 
# Turning point:  $x_t = x_0 E / V_0$ 
# Returns  $\psi$  from the left (0 to  $x_t$ ) and from the right ( $x_0$  to  $x_t$ )
def shoot(E):
    k = np.sqrt(2*(V0 - E))
    xt = x0*E/V0
    xl, ul, vl = rk4(E, 0, xt, 1, k)    # from the left
    xr, ur, vr = rk4(E, x0, xt, 1, -k) # from the right
    return xl, ul, vl, xr, ur, vr

# Both sides match at  $x_t$  when  $u_l'/u_l = u_r'/u_r$ 
# Written as  $f = u_l' * u_r - u_r' * u_l$  so we don't divide by zero
#  $f(E) = 0$  means  $E$  is an eigenvalue
def f(E):
    xl, ul, vl, xr, ur, vr = shoot(E)
    return vl*ur[-1] - vr*ul[-1]
```

Finding the eigenvalues

$f(E)$ changes sign at an eigenvalue. First I scan E in steps of 0.5 to find where the sign changes. Then I use bisection on that interval.

```
In [4]: # Bisection: keep the half where the sign changes until the interval is
smaller than tol
# a, b: interval where f changes sign
def bisection(a, b, tol=1e-10):
    while b - a > tol:
        m = (a + b)/2
        if f(a)*f(m) < 0: # sign change in the left half
            b = m
        else:             # sign change in the right half
            a = m
    return (a + b)/2

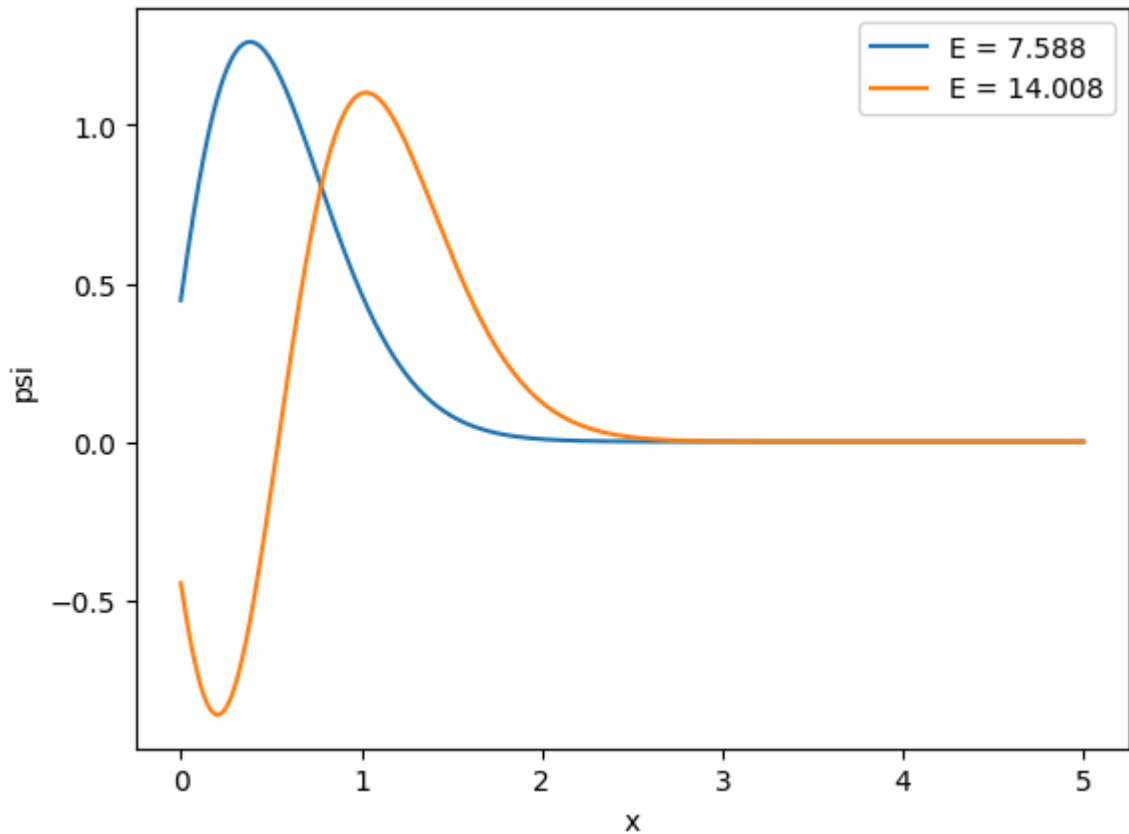
# Scan E from 0.5 to V0 (bound states have  $E < V_0$ )
# Stop after the two lowest eigenvalues
eigen = []
Es = np.arange(0.5, V0, 0.5)
for i in range(len(Es) - 1):
    if f(Es[i])*f(Es[i+1]) < 0: # sign change, so an eigenvalue in between
        eigen.append(bisection(Es[i], Es[i+1]))
    if len(eigen) == 2:
        break
print(eigen)
```

```
[np.float64(7.588273770787055), np.float64(14.00829759662156)]
```

The two lowest eigenvalues are $E_0 = 7.588$ a.u. and $E_1 = 14.008$ a.u.

Plotting the eigenfunctions

```
In [5]: # For each eigenvalue:
# Scale the left part so both parts meet at xt
# Join the left part and the right part (flipped, since it goes from x0 to
xt)
# Normalize so that the integral of psi^2 is 1
for E in eigen:
    xl, ul, vl, xr, ur, vr = shoot(E)
    ul = ul*ur[-1]/ul[-1]
    x = np.concatenate([xl, xr[::-1]])
    psi = np.concatenate([ul, ur[::-1]])
    psi = psi/np.sqrt(trapezoid(psi**2, x))
    plt.plot(x, psi, label=f"E = {E:.3f}")
plt.xlabel("x"); plt.ylabel("psi"); plt.legend()
plt.show()
```



The ground state has no node and the first excited state has one node. Both are near the left wall, where the potential is lowest.

Conclusion

The two lowest eigenvalues are $E_0 = 7.588$ a.u. and $E_1 = 14.008$ a.u.